



Week 5 Workshop

Indexing and Query

Processing

IFN647

School of Computer Science
Queensland University of Technology

Workshop Objectives

- Learn how to process a collection of documents in a folder using the glob module.
- Develop a Python function to construct an inverted index for a given set of documents.
- Understand how to perform query processing in Python.
- *Learn how to manage file paths for data access in Python (Python skills).*
 - *Assume the Jupyter Notebook is executed from the working directory.*
 - *Use functions from the os module to identify the current working directory (current_path) and construct the path to the data directory (data_path).*
 - *Note that after calling index_docs, the working directory may be modified; therefore, it is important to reset it back to the original working directory to ensure consistent file access.*

Tasks

- Task1 practice (15min) / solution (10min)
- Task2&3 practice (30min) / solution (10min)
- Task4 practice (10min) / solution (5min)

Workshop Task 1

Task 1 For the given XML documents (in [w5_data-1.zip](#)), design a python function *index_docs()* to index them (please remove stop words and index stems only). The returned index should be a dictionary {term:{docID1:freq1, DocID2:freq2}, ...}

The following are a sample of outcomes, where '6146' and '741299' are the document id, respectively.

```
Index = {'argentin': {'6146': 1}, 'bond': {'6146': 1}, 'slight': {'6146': 2}, 'higher':  
{ '6146': 1}, 'small': {'6146': 1}, 'technic': {'6146': 2}, 'bounc': {'6146': 2},  
'wednesday': {'6146': 1}, ..., 'time': {'741299': 1}, 'abl': {'741299': 1}, 'push':  
{ '741299': 1}, 'hard': {'741299': 1}, 'big': {'741299': 1}, 'third': {'741299': 1},  
'finish': {'741299': 1}, 'porsch': {'741299': 1}, 'franc': {'741299': 1}, 'bob':  
{ '741299': 1}, 'wollek': {'741299': 1}, 'yannick': {'741299': 1}, 'dalma': {'741299':  
1}, 'belgian': {'741299': 2}, 'thierri': {'741299': 1}, 'boutsen': {'741299': 1},  
'former': {'741299': 1}, 'formula': {'741299': 1}, 'one': {'741299': 1}, 'driver':  
{ '741299': 1}, 'switch': {'741299': 1}, 'normal': {'741299': 1}, 'share': {'741299':  
1}, 'han': {'741299': 1}, 'stuck': {'741299': 1}, 'follow': {'741299': 1}, 'power':  
{ '741299': 1}, 'steer': {'741299': 1}, 'failur': {'741299': 1}}
```

From previous week...

The return value of function `parse_doc`

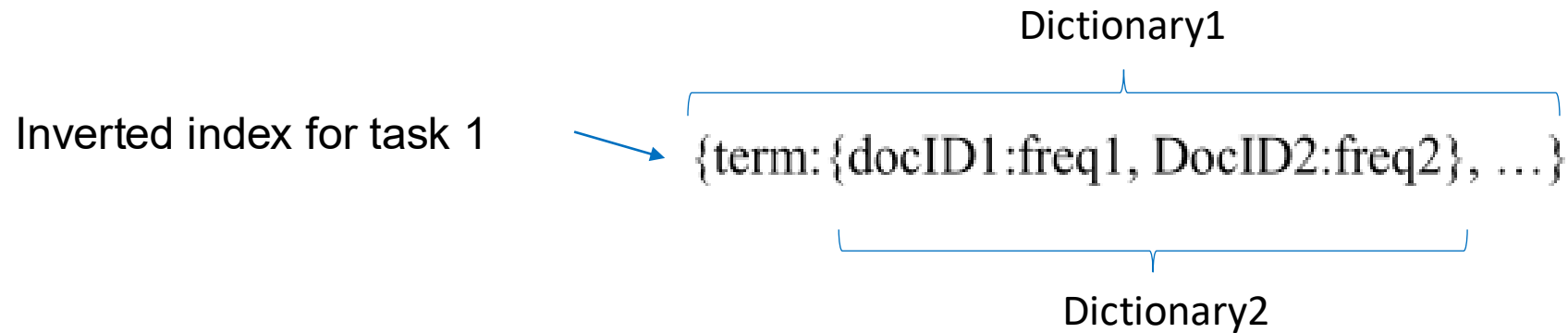
```
(133, {'6146': {'argentine': 1, 'bonds': 1, 'slightly': 1, 'higher': 1, 'small': 1, 'technical': 2, 'bounce': 2, 'wednesday': 1, 'amid': 1, 'low': 1, 'volume': 1, 'trader': 2, 'large': 1, 'foreign': 1, 'bank': 1, 'slight': 1, 'opening': 1, 'expect': 1, 'prices': 1, 'change': 1, 'much': 1, 'during': 1, 'session': 1, 'market': 2, 'moving': 1, 'news': 1, 'expected': 2, 'percent': 1, 'dollar': 1, 'denominated': 1, 'bocon': 1, 'previsional': 1, 'due': 2, 'rose': 2, 'argentina': 2, 'frb': 1, 'quot': 2, 'general': 1, 'uncertainty': 1, 'pointing': 1, 'events': 1, 'waiting': 1, 'including': 1, 'passage': 1, 'government': 1, 'new': 1, 'economic': 1, 'measures': 1, 'through': 1, 'congress': 1, 'now': 1, 'until': 1, 'early': 1, 'october': 1, 'addition': 1, 'traders': 1, 'awaiting': 1, 'meeting': 1, 'friday': 1, 'between': 1, 'economy': 1, 'minister': 1, 'roque': 1, 'fernandez': 1, 'international': 1, 'monetary': 1, 'fund': 1, 'delegation': 1, 'fiscal': 1, 'deficit': 1, 'axel': 1, 'bugge': 1, 'buenos': 1, 'aires': 1, 'newsroom': 1}})
```

Expected Output...

```
Index = {'argentin': {'6146': 1}, 'bond': {'6146': 1}, 'slight': {'6146': 2}, 'higher': {'6146': 1}, 'small': {'6146': 1}, 'technic': {'6146': 2}, 'bounc': {'6146': 2}, 'wednesday': {'6146': 1}, ..., 'time': {'741299': 1}, 'abl': {'741299': 1}, 'push': {'741299': 1}, 'hard': {'741299': 1}, 'big': {'741299': 1}, 'third': {'741299': 1}, 'finish': {'741299': 1}, 'porsch': {'741299': 1}, 'franc': {'741299': 1}, 'bob': {'741299': 1}, 'wollek': {'741299': 1}, 'yannick': {'741299': 1}, 'dalma': {'741299': 1}, 'belgian': {'741299': 2}, 'thierri': {'741299': 1}, 'boutsen': {'741299': 1}, 'former': {'741299': 1}, 'formula': {'741299': 1}, 'one': {'741299': 1}, 'driver': {'741299': 1}, 'switch': {'741299': 1}, 'normal': {'741299': 1}, 'share': {'741299': 1}, 'han': {'741299': 1}, 'stuck': {'741299': 1}, 'follow': {'741299': 1}, 'power': {'741299': 1}, 'steer': {'741299': 1}, 'failur': {'741299': 1}}
```

Workshop Task 1 Knowledge Hint – Inverted Index

- Indexes are **data structures** designed to make search faster.
- In this task, you're building an **inverted index**, which is a structure used in search engines like Google.
- It maps terms to the documents they occur in. Each term points to a list of document identifiers where that term appears, along with additional information such as term frequency, position, etc.



Workshop Task 1 Knowledge Hint – glob

The **glob** module in Python

The **glob** module in Python is used for file path expansion.
It helps find files and directories whose names match a specified pattern.

Example:

```
import glob  
glob.glob("*.xml")
```

is used to find all files in the current directory with an extension of ".xml".

- **glob.glob()** is a function from the **glob** module.
- **"*.xml"** is the pattern provided to **glob.glob()**. It specifies that we're looking for files with names ending with ".xml".
- **glob.glob("*.xml")** returns a list of file paths matching the specified pattern.

Workshop Task 2 & 3

Task 2. Design a python function *doc_at_a_time(I, Q)*, where index I is a Dictionary of term:Dictionary of (itemId:freq), which returns a dictionary of docId:relevance for the given query Q (a term:freq dictionary).

Input:

I - Inverted Index - {term: {docID1:freq1, DocID2:freq2}, ...}

Q – {'term1': freq1, 'term2': freq2, ...} where term1, term2, ... terms in a query.

Output:

{docID1: relevanceScore, docID2: relevanceScore }

Task 3. Design a python function *term_at_a_time(I, Q)*, where index I is a Dictionary of term:Dictionary of (itemId:freq), which returns a dictionary of docId:relevance for the given query Q (a term:freq dictionary).

Workshop Task 2&3 (Recap the abstraction model of ranking)

Relevance Score of a document (D) to a given query (Q)

$$R(Q, D) = \sum_i g_i(Q) f_i(D)$$

f_i is a document feature function

g_i is a query feature function

Workshop Task 2&3 Hint

Example: Query {salt:1, water:1, tropical:1}

Document-at-a-time

- Calculates complete scores for documents by processing all term lists, one document at a time.

salt	1:1			4:1
water	1:1	2:1		4:1
tropical	1:2	2:2	3:1	
score	1:4	2:3	3:1	4:2

Term-at-a-time

- Accumulates scores for documents by processing term lists one at a time.

salt	1:1	4:1		
partial scores	1:1	4:1		
old partial scores	1:1	4:1		
water	1:1	2:1	4:1	
new partial scores	1:2	2:1	4:2	
old partial scores	1:2	2:1	4:2	
tropical	1:2	2:2	3:1	
final scores	1:4	2:3	3:1	4:2

Workshop Task 4

Task 4. Design a python main program to call the above three functions for a query, e.g.,
Query = {'formula':1, 'one':1}.

The sample outputs:

Document_at_a_time result-----

Document ID: 741299 and relevance weight: 2

...

Term_at_a_time result -----

Document ID: 741299 and relevance weight: 2

...

Basic idea:

1. Obtain the stopping word list
2. Obtain Inverted Index :index_docs (data_path, stop_words)
3. Call doc_at_a_time(Index, Query)
4. Call term_at_a_time(Index, Query)
5. Ranking the result: sorted function
6. Retrieval each result and print the output